Generative Learning Algorithms: Focus on Gaussian Discriminant Analysis (GDA) and Naive Bayes

April 25, 2026

From Discriminative to Generative Thinking

Most classification algorithms directly estimate $p(y \mid x)$. In this view, the model draws a decision boundary in feature space and assigns a label based on which side of the boundary a new point lies. Models such as logistic regression and the perceptron are examples of **discriminative learning**.

Generative learning algorithms take a different perspective. They model the data generation process itself by learning:

- the class prior $p(y)$, and
- the class-conditional density $p(x \mid y)$.

Once these quantities are estimated, Bayes' rule gives the posterior:

$$p(y \mid x) = \frac{p(x \mid y)p(y)}{p(x)}.$$

For classification, we often compare $p(x \mid y)p(y)$ across classes and ignore the normalizing term $p(x)$.

A key advantage of generative models is that they can also simulate new examples. For example, to generate a synthetic example, we can first sample a class label y from $p(y)$ and then sample a feature vector x from $p(x \mid y)$.

Classical generative models include **Naive Bayes** and **Gaussian Discriminant Analysis (GDA)**. The next sections develop the mathematical foundations needed for GDA and then derive the model in full detail.

1 Multivariate Gaussians

A multivariate Gaussian distribution describes continuous vectors in $\mathbb{R}^n$.

Definition A random vector $x \in \mathbb{R}^n$ is Gaussian if it has density:

$$p(x; \mu, \Sigma) = \frac{1}{(2\pi)^{n/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)\right),$$

where $\mu \in \mathbb{R}^n$ is the mean and $\Sigma \in \mathbb{R}^{n\times n}$ is the covariance matrix.

We write:

$$x \sim \mathcal{N}(\mu, \Sigma).$$

$$\Sigma = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \qquad \Sigma = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix} \qquad \Sigma = \begin{pmatrix} 1 & 0.8 \\ 0.8 & 1 \end{pmatrix}$$

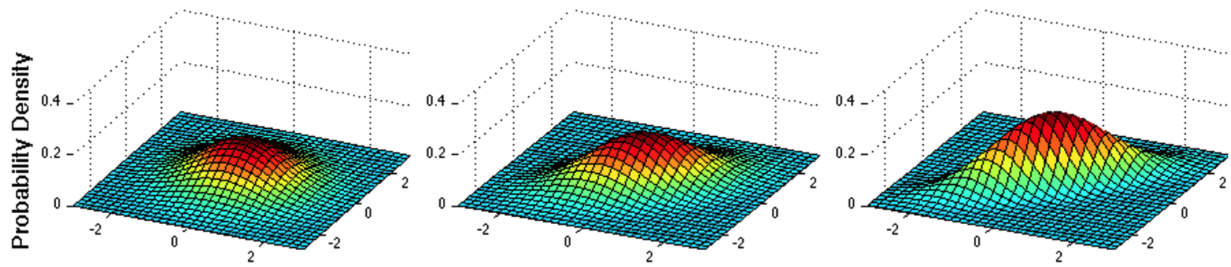

Figure: Probability density function

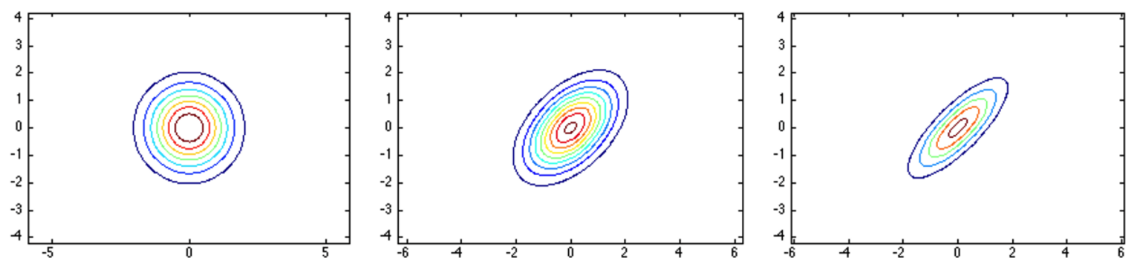

Figure: Contour plot of the pdf

Geometry and intuition

- μ locates the center of the distribution.
- Σ controls spread and correlation.
- In 2D, constant-density contours are ellipses; in higher dimensions they are ellipsoids.

The covariance entries are:

$$\mathrm{Cov}(x_i, x_j) = \mathbb{E}[(x_i - \mathbb{E}[x_i])(x_j - \mathbb{E}[x_j])] = [\Sigma]_{ij}.$$

For a vector Z, the covariance matrix can also be written as:

$$\Sigma = \mathbb{E}[(Z - \mu)(Z - \mu)^T] = \mathbb{E}[ZZ^T] - \mu\mu^T.$$

Useful Gaussian properties

Normalization The density integrates to 1:

$$\int_{\mathbb{R}^n} p(x; \mu, \Sigma)\, dx = 1.$$

Marginal distributions If we partition x and μ as $x = [x_A; x_B]$ and similarly partition Σ, then x_A is Gaussian with mean μ_A and covariance Σ_{AA}.

Conditional distributions If (x_A, x_B) is jointly Gaussian, then

$$x_A \mid x_B \sim \mathcal{N}\left(\mu_A + \Sigma_{AB}\Sigma_{BB}^{-1}(x_B - \mu_B),\ \Sigma_{AA} - \Sigma_{AB}\Sigma_{BB}^{-1}\Sigma_{BA}\right).$$

Sum of independent Gaussians If $y \sim \mathcal{N}(\mu_y, \Sigma_y)$ and $z \sim \mathcal{N}(\mu_z, \Sigma_z)$ are independent, then

$$y + z \sim \mathcal{N}(\mu_y + \mu_z, \Sigma_y + \Sigma_z).$$

2 Gaussian Discriminant Analysis

GDA models the data generation process for binary classification using a Gaussian distribution for each class.

2.1 Model assumptions

For binary labels $y \in \{0, 1\}$:

$$y \sim \text{Bernoulli}(\phi), \qquad x \mid y = k \sim \mathcal{N}(\mu_k, \Sigma), \qquad k \in \{0, 1\}.$$

The shared covariance matrix Σ means both classes have the same contour shape, but different centers.

2.2 Joint distribution and likelihood

The joint density is:

$$p(x, y) = p(x \mid y)p(y).$$

Given training data $\{(x^{(i)}, y^{(i)})\}_{i=1}^m$, the log-likelihood is:

$$\ell(\phi, \mu_0, \mu_1, \Sigma) = \sum_{i=1}^m \log p(x^{(i)}, y^{(i)}; \phi, \mu_0, \mu_1, \Sigma).$$

Substituting the Gaussian and Bernoulli models gives:

$$\ell = \sum_{i=1}^m \Big[-\frac{n}{2}\log 2\pi - \frac{1}{2}\log|\Sigma| - \frac{1}{2}(x^{(i)} - \mu_{y^{(i)}})^T\Sigma^{-1}(x^{(i)} - \mu_{y^{(i)}}) \\ + y^{(i)}\log\phi + (1 - y^{(i)})\log(1 - \phi)\Big].$$

2.3 Matrix derivative identities

Useful derivative formulas are:

$$\frac{\partial}{\partial x} x^T A x = (A + A^T)x,$$
$$\frac{\partial}{\partial X}\log|X| = (X^{-1})^T,$$

and for vectors a, b:

$$\frac{\partial}{\partial X} a^T X^{-1} b = -X^{-T} a b^T X^{-T}.$$

When X is symmetric, $X^{-T} = X^{-1}$.

2.4 Maximum likelihood estimation

Estimate for ϕ Only the Bernoulli terms depend on ϕ:

$$\ell_\phi = \sum_{i=1}^{m} \left[y^{(i)} \log \phi + (1 - y^{(i)}) \log(1 - \phi) \right].$$

Setting the derivative to zero:

$$\frac{\partial \ell_\phi}{\partial \phi} = \frac{1}{\phi} \sum_{i=1}^{m} y^{(i)} - \frac{1}{1-\phi} \sum_{i=1}^{m} (1 - y^{(i)}) = 0.$$

Solving gives:

$$\phi = \frac{1}{m} \sum_{i=1}^{m} y^{(i)}.$$

Estimate for μ_k The relevant term is:

$$\ell_{\mu_k} = -\frac{1}{2} \sum_{i:y^{(i)}=k} (x^{(i)} - \mu_k)^T \Sigma^{-1} (x^{(i)} - \mu_k).$$

Using $\partial_\mu (x-\mu)^T A (x-\mu) = -2A(x-\mu)$ for symmetric A, we obtain:

$$\frac{\partial \ell_{\mu_k}}{\partial \mu_k} = \sum_{i:y^{(i)}=k} \Sigma^{-1} (x^{(i)} - \mu_k) = 0.$$

Multiplying by Σ leads to:

$$\sum_{i:y^{(i)}=k} x^{(i)} = m_k \mu_k, \qquad m_k = \sum_{i=1}^{m} 1\{y^{(i)} = k\}.$$

Thus,

$$\mu_k = \frac{1}{m_k} \sum_{i:y^{(i)}=k} x^{(i)}.$$

Estimate for Σ The covariance term contributes both a log-determinant and a quadratic form. The derivative of the log-determinant part is:

$$\frac{\partial}{\partial \Sigma} \left(-\frac{m}{2} \log |\Sigma| \right) = -\frac{m}{2} \Sigma^{-1}.$$

The derivative of the quadratic term is:

$$\frac{\partial}{\partial \Sigma} \left(-\frac{1}{2} (x-\mu)^T \Sigma^{-1} (x-\mu) \right) = \frac{1}{2} \Sigma^{-1} (x-\mu)(x-\mu)^T \Sigma^{-1}.$$

Setting the derivative of ℓ to zero yields:

$$\frac{1}{2} \sum_{i=1}^{m} \Sigma^{-1} (x^{(i)} - \mu_{y^{(i)}})(x^{(i)} - \mu_{y^{(i)}})^T \Sigma^{-1} - \frac{m}{2} \Sigma^{-1} = 0.$$

Multiplying by 2Σ gives:

$$\sum_{i=1}^{m} (x^{(i)} - \mu_{y^{(i)}})(x^{(i)} - \mu_{y^{(i)}})^T = m\Sigma.$$

So the pooled covariance estimate is:

$$\Sigma = \frac{1}{m}\sum_{i=1}^{m}(x^{(i)} - \mu_{y^{(i)}})(x^{(i)} - \mu_{y^{(i)}})^T.$$

2.5 Summary of GDA estimates

$$\phi = \frac{1}{m}\sum_{i=1}^{m} y^{(i)}, \qquad \mu_k = \frac{1}{m_k}\sum_{i:y^{(i)}=k} x^{(i)}, \qquad \Sigma = \frac{1}{m}\sum_{i=1}^{m}(x^{(i)} - \mu_{y^{(i)}})(x^{(i)} - \mu_{y^{(i)}})^T.$$

3 Special Cases and Variations

3.1 Linear Discriminant Analysis

GDA with shared covariance is also known as Linear Discriminant Analysis (LDA). The common covariance causes the discriminant boundary to be linear in x.

3.2 Quadratic Discriminant Analysis

If we allow different covariance matrices for each class,

$$x \mid y = k \sim \mathcal{N}(\mu_k, \Sigma_k),$$

then the boundary is generally quadratic. The MLEs are:

$$\mu_k = \frac{1}{m_k}\sum_{i:y^{(i)}=k} x^{(i)}, \qquad \Sigma_k = \frac{1}{m_k}\sum_{i:y^{(i)}=k}(x^{(i)} - \mu_k)(x^{(i)} - \mu_k)^T.$$

3.3 Diagonal covariance

When Σ is diagonal, features are assumed independent given the class. This is analogous to Naive Bayes for continuous features.

3.4 Isotropic covariance

If $\Sigma = \sigma^2 I$, then all directions share the same variance and the density simplifies to:

$$p(x; \mu, \sigma^2) = \frac{1}{(2\pi\sigma^2)^{n/2}} \exp\left(-\frac{1}{2\sigma^2}\|x - \mu\|^2\right).$$

3.5 Equal means

If $\mu_0 = \mu_1$, the model cannot separate classes using features; only the prior ϕ remains useful.

4 Relation to Logistic Regression

Shared covariance GDA yields a posterior of logistic form.

4.1 Proof

By Bayes' rule,

$$p(y = 1 \mid x) = \frac{p(x \mid y = 1)\phi}{p(x \mid y = 1)\phi + p(x \mid y = 0)(1 - \phi)}.$$

The log-odds are:

$$\log \frac{p(y = 1 \mid x)}{p(y = 0 \mid x)} = \log \frac{p(x \mid y = 1)}{p(x \mid y = 0)} + \log \frac{\phi}{1 - \phi}.$$

Using the Gaussian densities and canceling constant terms leads to:

$$\log \frac{p(y = 1 \mid x)}{p(y = 0 \mid x)} = x^T \Sigma^{-1}(\mu_1 - \mu_0) - \frac{1}{2}(\mu_1^T \Sigma^{-1} \mu_1 - \mu_0^T \Sigma^{-1} \mu_0) + \log \frac{\phi}{1 - \phi}.$$

Define:

$$\theta = \Sigma^{-1}(\mu_1 - \mu_0), \qquad \theta_0 = \log \frac{\phi}{1 - \phi} - \frac{1}{2}(\mu_1^T \Sigma^{-1} \mu_1 - \mu_0^T \Sigma^{-1} \mu_0).$$

Then

$$p(y = 1 \mid x) = \frac{1}{1 + \exp(-\theta^T x - \theta_0)}.$$

5 Naive Bayes and Text Classification

So far, GDA assumes that the feature vectors x are continuous, real-valued vectors. A different generative model is needed when the features are discrete-valued. A motivating example is email spam filtering. In this problem, we want to classify messages as unsolicited commercial email (spam) or non-spam email. After training, a mail reader can automatically filter spam into a separate folder.

Text classification is a common application of discrete generative models. The key difference from GDA is the way we represent an example as a feature vector.

5.1 The multivariate Bernoulli event model

In the multivariate Bernoulli model, we represent an email by a binary vector whose length equals the vocabulary size. If the message contains the j-th vocabulary word, then we set:

$$x_j = 1, \qquad \text{otherwise } x_j = 0.$$

For example, if the vocabulary includes the words `a`, `aardvark`, `aardwolf`, ..., `buy`, ..., `zygmurgy`, then the vector $x = [1, 0, 0, \ldots, 1, \ldots, 0]^T$ represents an email containing `a` and `buy` but not `aardvark`, `aardwolf`, or `zygmurgy`.

The vocabulary is the set of words encoded into the feature vector. If the vocabulary has size V, then $x \in \{0, 1\}^V$.

Modeling $p(x \mid y)$ directly would require a massive number of parameters. For example, if $V = 50000$, then x is a 50000-dimensional binary vector. A full joint distribution over all possible vectors has 2^{50000} outcomes, which is impossible to estimate.

To make the problem tractable, Naive Bayes assumes that the features are conditionally independent given the class label. This is the **Naive Bayes assumption**:

$$p(x_j \mid y, x_{\neg j}) = p(x_j \mid y) \qquad \text{for all } j,$$

where $x_{\neg j}$ denotes all features except x_j. This means that once we know the class y, knowledge of one word's presence does not change the probability of another word's presence.

Note that this is weaker than unconditional independence. We are not assuming $p(x_j) = p(x_j, x_k)$; we assume independence only after conditioning on y.

Using the chain rule and the Naive Bayes assumption:

$$\begin{aligned} p(x \mid y) &= p(x_1 \mid y)\, p(x_2 \mid y, x_1)\, p(x_3 \mid y, x_1, x_2) \cdots p(x_V \mid y, x_1, \ldots, x_{V-1}) \\ &= \prod_{j=1}^{V} p(x_j \mid y). \end{aligned}$$

Even though the assumption is very strong, Naive Bayes often performs well in practice.

Model parameters The model is parameterized by:

$$\phi = p(y = 1), \qquad \phi_{j|1} = p(x_j = 1 \mid y = 1), \qquad \phi_{j|0} = p(x_j = 1 \mid y = 0).$$

Given training data $(x^{(i)}, y^{(i)})_{i=1}^m$, the maximum likelihood estimates are:

$$\phi = \frac{1}{m} \sum_{i=1}^{m} y^{(i)},$$

$$\phi_{j|1} = \frac{\sum_{i=1}^m 1\{x_j^{(i)} = 1 \wedge y^{(i)} = 1\}}{\sum_{i=1}^m 1\{y^{(i)} = 1\}}, \qquad \phi_{j|0} = \frac{\sum_{i=1}^m 1\{x_j^{(i)} = 1 \wedge y^{(i)} = 0\}}{\sum_{i=1}^m 1\{y^{(i)} = 0\}}.$$

For example, $\phi_{j|1}$ is the fraction of spam emails in which word j appears.

Prediction For a new email represented by x, we compute:

$$p(y = 1 \mid x) = \frac{p(x \mid y = 1)p(y = 1)}{\sum_{k \in \{0,1\}} p(x \mid y = k)p(y = k)}$$

with

$$p(x \mid y = k) = \prod_{j=1}^{V} p(x_j \mid y = k)$$

because of the Naive Bayes assumption.

5.2 Multinomial event model

A second text model represents an email as a sequence of words rather than as a binary presence vector. Suppose the email has d words, and each word x_j is an index into the vocabulary of size V. Then $x = (x_1, \ldots, x_d)$ with each $x_j \in \{1, \ldots, V\}$.

If we modeled the full joint distribution of these word sequences explicitly, the number of possible outcomes would be enormous. Instead, Naive Bayes assumes that the word positions are conditionally independent given y, and that each word is drawn from the same class-dependent distribution.

Under this model,

$$p(x \mid y) = \prod_{j=1}^{d} p(x_j \mid y),$$

where

$$\theta_{k,v} = p(x_j = v \mid y = k)$$

is the probability of word v in class k.

Given training data, the MLEs are:

$$\theta_{k,v} = \frac{\sum_{i=1}^{m} \sum_{j=1}^{d^{(i)}} 1\{x_j^{(i)} = v \wedge y^{(i)} = k\}}{\sum_{i=1}^{m} 1\{y^{(i)} = k\}\, d^{(i)}},$$

and

$$\phi = \frac{1}{m} \sum_{i=1}^{m} y^{(i)}.$$

This is the multinomial event model for text classification.

5.3 Discretizing continuous attributes

If an input feature is originally continuous, it can still be used with Naive Bayes by discretizing its range. For example, living area might be encoded as:

Living area (sq. ft.)	Feature value
¡ 400	1
400–800	2
800–1200	3
1200–1600	4
¿ 1600	5

Then the Naive Bayes model treats the discretized value as a categorical feature and estimates a multinomial probability for it.

5.4 Laplace smoothing

A useful enhancement is Laplace smoothing. Without it, if a word never appears in the training data for a given class, the estimated probability becomes zero. This causes the class posterior to become zero for any test example containing that word.

For example, if the word `neurips` never occurred in the training examples, the model would estimate $p(\texttt{neurips} \mid y = 1) = 0$ and $p(\texttt{neurips} \mid y = 0) = 0$, so the posterior becomes undefined.

To avoid this, Laplace smoothing adds one pseudo-count to every category. For the binary model,

$$\phi_{j|k} = \frac{1 + \sum_{i=1}^{m} 1\{x_j^{(i)} = 1 \wedge y^{(i)} = k\}}{2 + \sum_{i=1}^{m} 1\{y^{(i)} = k\}}.$$

For the multinomial event model,

$$\theta_{k,v} = \frac{1 + \sum_{i=1}^{m} \sum_{j=1}^{d^{(i)}} 1\{x_j^{(i)} = v \wedge y^{(i)} = k\}}{V + \sum_{i=1}^{m} 1\{y^{(i)} = k\}\, d^{(i)}},$$

where V is the vocabulary size.

Laplace smoothing ensures that no probability is exactly zero and that the parameter estimates still sum to one.

6 Conclusion

Generative learning algorithms provide a principled way to model the joint distribution of features and labels. GDA is a powerful example that combines Gaussian assumptions with Bayes' theorem. When the shared-covariance assumption is valid, GDA leads to a linear classifier and has a direct connection to logistic regression. More flexible variants such as QDA trade this simplicity for richer decision boundaries.

References

References

[1] Andrew Ng. *CS229: Machine Learning – Lecture Notes 2 (Generative Learning Algorithms)*. Stanford University, Spring 2019. https://cs229.stanford.edu/notes-spring2019/cs229-notes2.pdf

[2] Wei Zhang. *Generative Learning Algorithm*. Machine Learning Hub. https://wei2624.github.io/MachineLearning/sv_generative_model/

[3] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Second Edition. Springer, 2009. Chapter 4: Linear Methods for Classification.

[4] Kevin P. Murphy. *Machine Learning: A Probabilistic Perspective*. MIT Press, 2012. Chapter 8: Logistic Regression.

[5] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning*. Second Edition. Springer, 2021. Chapter 4: Classification.

[6] Jesse Lucas. *CSC 411: Machine Learning – Lecture 9 Handout*. University of Toronto, Computer Science Department. https://www.cs.toronto.edu/~jlucas/teaching/csc411/lectures/lec9_handout.pdf

[7] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. Chapter 5: Machine Learning Basics.

[8] J. Pennington, R. Socher, and C. D. Manning. *GloVe: Global Vectors for Word Representation*. EMNLP 2014. https://nlp.stanford.edu/projects/glove/

[9] David D. Lewis. *Naive (Bayes) at Twenty: Twenty Years of Text Classification*. ACM SIGIR Forum, 2004.